Clustering Incrémental Binaire par MDL et Fusion par Préfixe Commun

Abstract

Nous présentons un schéma de clustering incrémental pour données numériques converties en mots binaires de longueur fixe L. La méthode comporte deux étapes principales :

1. attribution incrémentale des instances à des prototypes via un critère Minimum Description Length (MDL) local,
2. fusion globale de prototypes similaires à l'aide d'un coefficient d'accuité α sur la longueur de leur préfixe commun, sans recourir à une métrique continue. L'accuité est enfaite un zoom sur les clusters, plus l'acuité est grande plus on a des clsuters généraux, plus l'accuité est faible plus les clusters sont précis.

Cette approche allie frugalité algorithmique — pas de recalcul global à chaque étape — et paramétrisation minimale.

1 Introduction

Nous exploitons la représentation binaire des valeurs pour construire des *prototypes* compacts et utilisons un critère MDL pour décider, à l'arrivée de chaque instance, s'il faut la rattacher à un prototype existant ou en créer un nouveau. Enfin, un second passage de fusion regroupe les prototypes jugés semblables selon leur *préfixe commun* majoritaire.

2 Clustering incrémental par MDL local

2.1 Représentation des prototypes

Chaque *prototype* P_j est défini par :

$$P_j = (b_j,\ O_j = \{\, o_{j,1}, \ldots, o_{j,n_j} \}), \quad b_j \in \{0,1\}^L,\ o_{j,k} \in \{0,1\}^L,$$

avec pour toute instance $x_{j,k}$ du cluster :

$$x_{j,k} = b_j \oplus o_{j,k}.$$

2.2 Critère MDL pour le rattachement

À l'arrivée d'une nouvelle instance $x \in \{0,1\}^L$, deux opérations sont comparées :

1. **Rattacher** x au prototype P_j existant,
2. **Créer** un nouveau prototype dont la base est x.

On note :

$$n_j = |O_j|, \quad b_j^{\text{old}} = \max_{o \in O_j} (\text{nombre de bits pour coder } o),$$

et

$$b_j^{\text{new}} = \max(b_j^{\text{old}}, \text{ bits pour coder } x \oplus b_j).$$

Le *coût* description *avant* rattachement vaut

$$L_{\text{before}} = L + n_j \, b_j^{\text{old}},$$

et le coût *après* rattachement

$$L_{\text{after}} = L + (n_j + 1) \, b_j^{\text{new}}.$$

Créer un nouveau prototype pour x coûte en plus L bits. On définit alors la variation

$$\Delta L_j = L_{\text{after}} - (L_{\text{before}} + L) = (n_j + 1) \, b_j^{\text{new}} - n_j \, b_j^{\text{old}} - L.$$

Si $\Delta L_j \leq 0$, alors rattacher à P_j, sinon créer un nouveau prototype.

Exemple numérique

Pour $L = 4$, prototype

$$P_j : \quad b_j = \texttt{1010}, \quad O_j = \{\texttt{0000}, \texttt{0011}\},$$

donc $b_j^{\text{old}} = 2$ bits (pour coder `11`). Pour $x = \texttt{1001}$:

$$x \oplus b_j = \texttt{0011}, \quad b_j^{\text{new}} = 2.$$

Avec $n_j = 2$:

$$L_{\text{before}} = 4 + 2 \times 2 = 8, \quad L_{\text{after}} = 4 + 3 \times 2 = 10,$$

$$\Delta L_j = 10 - (8 + 4) = -2 \leq 0.$$

On rattache donc x au prototype existant.

2.3 Algorithme incrémental

Algorithm 1 Clustering incrémental MDL

Require: flux d'instances binaires $\{x_t\}_{t=1}^{N}$, longueur L

Initialiser liste de prototypes $\mathcal{C} \leftarrow \emptyset$
for $t = 1, \ldots, N$ **do**
  Pour chaque $P_j \in \mathcal{C}$ calculer ΔL_j
  if $\exists j^*$ tel que $\Delta L_{j^*} \leq 0$ **then**
    Rattacher x_t à P_{j^*} et mettre à jour (b_{j^*}, O_{j^*})
  else
    Créer $P_{\text{new}} = (x_t, \{0^L\})$, ajouter à $\mathcal{C}$
  end if
end for
return $\mathcal{C}$

3 Indexation par trie de subsomption

Pour éviter de tester tous les prototypes à chaque étape, on maintient un *trie* binaire de profondeur L dont chaque nœud stocke la liste `cluster_idxs` des indices de prototypes partageant le préfixe associé.

3.1 Insertion d'une base b_j

- On descend depuis la racine (préfixe vide).
- À chaque nœud, on ajoute l'indice j à `cluster_idxs`.
- On suit (ou crée) l'enfant correspondant au bit courant (0 ou 1).

Illustration (4 bits) Insérons $b_0 =$ `1010` (cluster 0) puis $b_1 =$ `1001` (cluster 1) :

Étape	Préfixe parcouru	`cluster_idxs` mis à jour
racine	`""`	[0] → [0,1]
bit 1	`"1"`	[0] → [0,1]
bit 0	`"10"`	[0] → [0,1]
bit 1	`"101"`	[0] → -
bit 0/1	`"1010"`, `"1001"`	[0], [1]

3.2 Requête pour une instance x

- On descend bit à bit selon la chaîne x.
- On suit les enfants tant que possible, en copiant la liste `cluster_idxs`.
- Dès qu'un bit ne correspond à aucun enfant, on arrête : cette liste est l'ensemble des *candidats* à tester en MDL.

Exemple Pour $x = \texttt{1011}$, on parcourt successivement les préfixes `""`, `"1"`, `"10"`, `"101"` et on récupère les indices ‘[0]‘ au dernier nœud atteint — au lieu de tester tous les prototypes.

4 Fusion globale par préfixe commun (détail)

Après avoir construit l'ensemble initial de prototypes $\{P_j = (b_j, O_j)\}_{j=1}^{J}$, on réalise une fusion *globale* en trois étapes :

4.1 1. Calcul du préfixe commun maximal

Pour chaque prototype P_j, on extrait l'ensemble complet de ses membres en clair :

$$M_j \;=\; \{\, b_j \oplus o \;:\; o \in O_j \} \subset \{0,1\}^L.$$

On calcule alors son *préfixe commun maximal*

$$p_j = \mathrm{prefix}\big(M_j\big) = \max\{\, \ell \in \{0,\dots,L\} \mid \forall x, y \in M_j,\; x_{1:\ell} = y_{1:\ell}\},$$

où $x_{1:\ell}$ désigne les ℓ premiers bits de x.

4.2 2. Seuil de fusion $k = \lceil \alpha L \rceil$

On fixe un paramètre unique $\alpha \in (0,1)$ qui détermine la *précision* du regroupement. On pose

$$k = \lceil \alpha L \rceil.$$

Deux prototypes P_i et P_j sont considérés assez proches pour fusion s'ils partagent

$$p_i \geq k \quad \text{et} \quad p_j \geq k \quad \text{et} \quad b_i[1:k] = b_j[1:k].$$

4.3 3. Regroupement et création de nouveaux prototypes

On regroupe les indices

$$G_\sigma = \{\, j \mid b_j[1:k] = \sigma \} \quad \text{pour chaque préfixe } \sigma \in \{0,1\}^k.$$

Pour chaque groupe non vide G_σ, on crée un nouveau prototype

$$P^{(\sigma)} \;=\; \Big(\, b^{(\sigma)} \,,\; \bigcup_{j \in G_\sigma} O_j \Big),$$

où sa base $b^{(\sigma)}$ est choisie comme le *plus petit* des b_j, $j \in G_\sigma$, afin de minimiser les offsets :

$$b^{(\sigma)} = \min_{j \in G_\sigma} b_j \quad \text{(ordre lexicographique)}.$$

Algorithm 2 Fusion par préfixe commun

Require: Prototypes $\{(b_j, O_j)\}_{j=1}^{J}$, seuil α

$k \leftarrow \lceil \alpha L \rceil$
Initialiser un dictionnaire `groups`
for $j = 1$ **à** J **do**
    $\sigma \leftarrow b_j[1:k]$
    `groups`$[\sigma]$.append(j)
end for
$\mathcal{C}_{\text{fused}} \leftarrow \emptyset$
for chaque σ clé de `groups` **do**
    $G \leftarrow$ `groups`$[\sigma]$
    $b^{(\sigma)} \leftarrow \min_{j \in G} b_j$
    $O^{(\sigma)} \leftarrow \bigcup_{j \in G} O_j$
    Ajouter $(b^{(\sigma)}, O^{(\sigma)})$ à $\mathcal{C}_{\text{fused}}$
end for
return $\mathcal{C}_{\text{fused}}$

4.4 Pseudocode de la fusion

4.5 Complexité de la fusion

- Calcul des préfixes p_j et extraction des $\sigma_j = b_j[1:k]$: $O(J\,L)$.
- Regroupement par clé dans une table de hachage : $O(J)$.
- Création des nouveaux prototypes (union des offsets) : $O(\sum_j |O_j|)$.

Au total, la fusion s'effectue en $O(J\,L + \sum_j |O_j|)$, souvent dominé par $O(J\,L)$.

En combinant ce second passage avec l'étape incrémentale MDL, on obtient un schéma complémentaire où l'on affine *globalement* la granularité des prototypes sans back-tracking ni paramètre complexe supplémentaire (seul α).

5 Complexité

- Conversion et insertion/requête dans le trie : $O(L)$ par instance,
- Nombre de candidats MDL testé : $C \ll J$ en pratique, coût $O(C)$,
- Total amorti par instance : $O(L + C)$,
- Pour N instances : $O\big(N\,(L + C_{\max})\big)$.

6 Conclusion

Nous avons décrit un algorithme de clustering incrémental qui :

1. traite chaque donnée en temps constant amorti,
2. équilibre généralité des prototypes et fidélité par un critère MDL local,
3. indexe les prototypes dans un trie pour réduire drastiquement le nombre de comparaisons,